EE5203 Lab 7 Guide

PWM and input capture: dim LD2, then measure the signal back - Basri Erdogan

Mission: Dim LD2 with a TIM2 PWM channel and step its brightness with button B1, with no `HAL_Delay()` anywhere. Then connect PA5 to PB6 with one jumper, measure the same signal with TIM4 input capture, and show that the measured period and duty match the values you set.

Prepare before class

- ☐ Bring your board, a USB **data** cable and **one male-to-male jumper wire**.
- ☐ Do the **Lab 7 pre-lab quiz** on CATS (two attempts; the higher score counts).
- ☐ Re-read the theory slides up to "What each TIM4 capture setting gives you."
- ☐ Compute from 16 MHz: which `PSC`, `ARR` and `CCR1` give 1 kHz PWM at 12 %?
- ☐ Know who writes `CCR1` in PWM mode and who writes it in input capture mode.

Learning objectives

By checkoff time, you can:

1. hand PA5 to TIM2 and generate PWM from the real 16 MHz clock;
2. change the duty from code and from the SFR view;
3. capture rising and falling edges on one pin with two TIM4 channels;
4. turn capture timestamps into period, frequency and duty, wrap-safe.

Hardware and files

Item	Value
Board	Nucleo-F401RE: LD2 is on PA5, which is also `TIM2_CH1`
Clock	**HSI, PLL off** (CubeMX step 2): timer clock = **16 MHz**
Generator	TIM2: 1 ms tick **and** PWM on channel 1, `PSC` 15, `ARR` 999
Button	B1 on PC13 (`GPIO_EXTI13`, as in Lab 5)
Analyzer	TIM4 CH1 + CH2 on PB6, `PSC` 15 (1 µs per count), `ARR` 65535
Wiring	from Checkpoint C on: **one jumper, D13 → D10** (figure below)
Tools	STM32CubeMX (new project) + VS Code
Project name	`EE5203_L07_<studentnumber>`
File you edit	`main.c`, only inside the `USER CODE` regions

Configure in CubeMX

Start of the session: if `C:\Work\<studentnumber>` does not contain your Lab 6 project, copy **all** your project folders from `H:\EE_Labs` into it first.

1. **STM32CubeMX:** Board Selector → **NUCLEO-F401RE** → accept the default initialization.
2. **Clock Configuration tab: System Clock Mux** = **HSI**; **AHB**, **APB1** and **APB2** prescalers = **/1**. Every box on the right reads **16**, as in Lab 6.
3. **PA5:** click the pin and choose `TIM2_CH1`. It replaces the `GPIO_Output` (LD2) setting. Do this before step 4, or CubeMX may put channel 1 on PA0.
4. **Timers → TIM2:** Clock Source = `Internal Clock`; Channel1 = `PWM Generation CH1`. Parameter Settings: Prescaler `15`, Counter Period `999`; PWM Generation Channel 1: Mode `PWM mode 1`, Pulse `0`.
5. **Timers → TIM4:** Clock Source = `Internal Clock`; Channel1 = `Input Capture direct mode`; Channel2 = `Input Capture indirect mode`. Prescaler `15`, Counter Period `65535`. Input Capture Channel 1: Polarity **Rising Edge**. Input Capture Channel 2: Polarity **Falling Edge**. Check that **PB6** is now green as `TIM4_CH1`.
6. **System Core → NVIC**: enable **TIM2 global interrupt**, **TIM4 global interrupt** and **EXTI line[15:10] interrupts** (B1 on PC13 stays `GPIO_EXTI13`).
7. Project Manager: name `EE5203_L07_<studentnumber>`, location `C:\Work\<studentnumber>`, Toolchain/IDE = **CMake** → Generate Code.
8. **Copy `C:\Users\Public\EE5203_template\.vscode` into the project folder**, as in every lab. (Own laptop: use **Set Up STM32Cube project(s)**, as in Lab 5.)
9. **Verify, do not add:** in `main.c`, `MX_TIM2_Init()` with `TIM_OCMODE_PWM1` and `MX_TIM4_Init()` with `TIM_ICSELECTION_DIREC` and `..._INDIRECTTI`; in `stm32f4xx_hal_msp.c`, `GPIO_AF1_TIM2` (PA5) and `GPIO_AF2_TIM4` (PB6); in `stm32f4xx_it.c`, `TIM2_IRQHandler()`, `TIM4_IRQHandler()` and `EXTI15_10_IRQHandler()`.

Checkpoint A - LD2 dimmed by PWM

TIM2 does two jobs from the same `PSC` and `ARR`: its update event is the 1 ms tick of Lab 6, and its channel 1 makes a 1 kHz PWM on PA5. Three additions:

1. **In `USER CODE BEGIN PV`**, the millisecond counter, as in Lab 6:

```
volatile uint32_t g_ms = 0;
```

2. **In `USER CODE BEGIN 4`**, the tick callback:

```
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    g_ms++;                                /* TIM2 update: every 1 ms */
}
```

   Every callback in this lab has exactly one source, so none of them checks which timer or pin called it. A second timer or button would need that check.

3. **In `USER CODE BEGIN 2`**, start both jobs and set the duty:

```
HAL_TIM_Base_Start_IT(&htim2);                      /* 1 ms tick: g_ms   */
HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_1);           /* PWM on PA5 (LD2)  */
TIM2->CCR1 = 120;                                   /* 12 % duty         */
```

4. Build (**Ctrl+Shift+B**), start the debugger (**F5**), Continue.

Evidence for Checkpoint A: LD2 glows, dimmer than full on. Pause; in the SFR view `TIM2->PSC` = 15, `TIM2->ARR` = 999, `TIM2->CCR1` = 120. Write **50** into `TIM2->CCR1` and Continue: LD2 is almost dark. Then write **1000**: full on (`CCR = ARR + 1`). No code changed. `GPIOA->MODER` bits 11:10 read `10`: PA5 belongs to the timer now. `g_ms` grows between pauses.

Checkpoint B - Brightness levels with B1

Each press of B1 moves LD2 to the next brightness level in a table. The button interrupt is the Lab 5 one; the 50 ms guard uses `g_ms`. Three additions:

1. **In `USER CODE BEGIN PV`**, below `g_ms`, the press flag and the level table:

```
volatile uint8_t  button_event  = 0;
volatile uint32_t last_press_ms = 0;
const uint16_t level[5] = { 0, 30, 120, 350, 1000 };  /* fixed CCR1 values, never written */
```

2. **In `USER CODE BEGIN 4`**, next to the tick callback, the button callback:

```
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    if ((g_ms - last_press_ms) >= 50U) {  /* B1: one press per 50 ms */
        last_press_ms = g_ms;
        button_event = 1;
    }
}
```

3. **In `USER CODE BEGIN 2`**, below the three start lines, the level index; **in `USER CODE BEGIN 3`**, the step:

```
/* USER CODE BEGIN 2 */
uint32_t idx = 2;                     /* level[2] = 120, as in Checkpoint A */

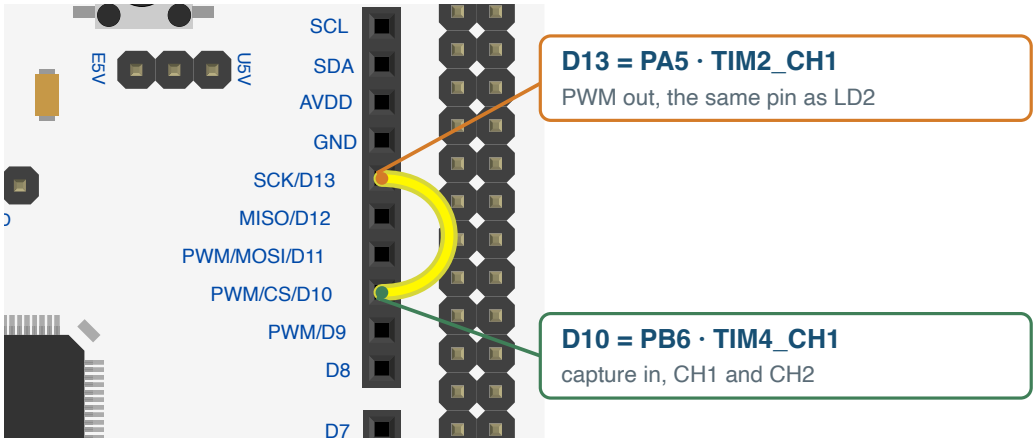
/* USER CODE BEGIN 3 */
if (button_event) {
    button_event = 0;
    idx = (idx + 1) % 5;
    TIM2->CCR1 = level[idx];
}
```

TIM2 holds the brightness between presses; the code runs only after a press.

Evidence for Checkpoint B: each press moves LD2 one level, 120 → 350 → 1000 → 0 → 30 → 120, and `TIM2->CCR1` in the SFR view shows the same values. No `HAL_Delay()` in `main.c`. **Predict, then check:** change the table to `{ 0, 250, 500, 750, 1000 }`. Which press changes the brightness most? Write one sentence: why does the first table look more even? Then put the first table back. **Screenshot `B_levels.png`:** the level table and your `USER CODE BEGIN 3` lines.

Checkpoint C - Captures arrive

Fit the jumper from **D13** (PA5, the PWM output) to **D10** (PB6, the capture input). Both pins are on the same right-hand header, three pins apart:



Three additions:

1. **In `USER CODE BEGIN PV`**, the results and the last rising timestamp:

```
volatile uint16_t period_us = 0, high_us = 0;
uint16_t last = 0;
```

2. **In `USER CODE BEGIN 2`**, below the PWM lines, start both capture channels. Only channel 1 gets an interrupt; channel 2 just captures:

```
HAL_TIM_IC_Start_IT(&htim4, TIM_CHANNEL_1);   /* rising,  direct:   interrupt    */
HAL_TIM_IC_Start(&htim4, TIM_CHANNEL_2);      /* falling, indirect: capture only */
```

3. **In `USER CODE BEGIN 4`**, next to the tick callback, the capture callback. It runs once per period, on each rising edge. At that moment `CCR2` still holds the falling edge just before it, and `last` holds the rising edge before that:

```
void HAL_TIM_IC_CaptureCallback(TIM_HandleTypeDef *htim)
{
    uint16_t now = TIM4->CCR1;                    /* this rising edge     */
    period_us = (uint16_t)(now - last);           /* rising to rising     */
    high_us   = (uint16_t)(TIM4->CCR2 - last);    /* rising to falling    */
    last = now;
}
```

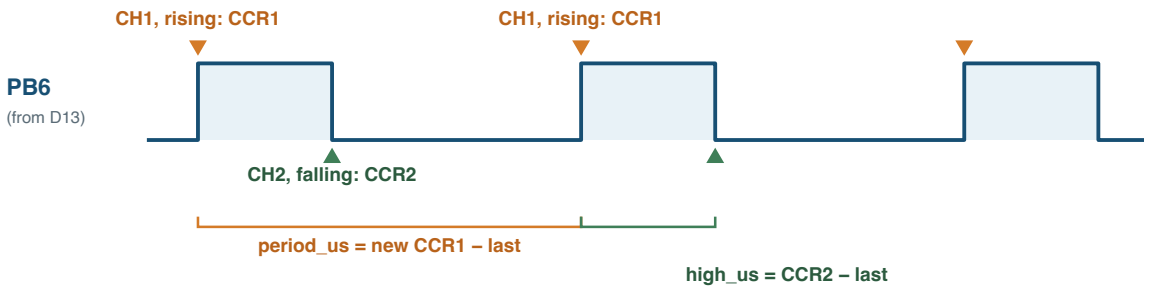
   No `if` is needed: TIM4 channel 1 is the only capture interrupt in this project. The `uint16_t` casts keep both subtractions right when `CNT` wraps.

Evidence for Checkpoint C: a breakpoint on `last = now;` is hit. Remove it, Continue, pause twice: `TIM4->CCR1` differs between the pauses, and no line of your code wrote it. The hardware timestamps every edge.

See the result live: in `.vscode/launch.json`, below `"runToEntryPoint": "main",` paste the line `"liveWatch": { "enabled": true, "samplesPerSecond": 4 },` then restart the debugger (Shift+F5, F5) and Continue. In **Run and Debug**, open **Cortex Live Watch** (own laptop: **STM32Cube Live Watch**), click +, add `period_us` and `high_us`. They update while the board runs: **1000** and **120** at level 120, `high_us` = 350 after one B1 press. Never read them at a breakpoint: the timers run on while the CPU is stopped. Pull the jumper out: the values freeze, because no capture arrives.

Checkpoint D - Measure period and duty

This is what the two capture channels see on PB6:



Fill the **Predicted** column first. Read the values in Live Watch, as in Checkpoint C.

1. Press B1 until LD2 is at the 350 level (or pause, write **350** into `TIM2->CCR1` in the SFR view and Continue). Read `period_us` and `high_us`.
2. Pause, write **1999** into `TIM2->ARR` in the SFR view, Continue, read again.
3. Pause, write **999** back into `TIM2->ARR`, then **0** into `TIM2->CNT`, Continue. (`CNT` may already be past 999; from there it would never meet the new `ARR`.) Press B1 through the levels and read `high_us` at each one. What do `period_us` and `high_us` show at level 0 and at level 1000? Why?

Quantity	Predicted	Measured
`period_us` with `ARR` = 999		
`high_us` with `CCR1` = 350		
duty (%) = `high_us` ÷ `period_us` × 100		
`period_us` with `ARR` = 1999		
duty (%) with `ARR` = 1999		

Evidence for Checkpoint D: the measured values match your prediction within ±2 µs, and you can explain the step 2 numbers. Copy the filled table into `USER CODE BEGIN Header` in `main.c` as a comment. **Screenshot `D_measure.png`:** `period_us` and `high_us` in Live Watch, and `TIM4->CCR1`, `TIM4->CCR2` in the SFR view.

Individual extension - Overcapture or a servo (optional, click)

Optional. Open this only after Checkpoint D is shown and time remains. Pick one.

- **Overcapture.** The timers keep counting while the debugger has stopped the CPU. Pause for a few seconds, Continue, pause again: `TIM4->SR` shows `CC1OF` set. Explain exactly which timestamp was lost, and why the callback could not read it.
- **Servo** (if your kit has one). TIM3 CH1 on **PA6 (D12)**, `PSC` 15, `ARR` 19999: a 20 ms frame, 50 Hz. Wire the servo signal to D12, its power to 5V and GND. `CCR` is the pulse width in µs: command 500, 1500 and 2500 (0°, 90°, 180°). If the servo buzzes at 500 or 2500, it is at its stop: move `CCR` 50 at a time toward 1500 until it is quiet. That value is your end point. Why can the servo not share TIM2?

Acceptance checklist

- ☐ `.ioc`: HSI 16 MHz; PA5 `TIM2_CH1`, PWM mode 1, `PSC` 15 / `ARR` 999; TIM4 CH1 direct rising, CH2 indirect falling, `PSC` 15 / `ARR` 65535; TIM2, TIM4 and EXTI line[15:10] enabled in NVIC.
- ☐ SFR evidence for Checkpoint A: `CCR1` changed by hand, `MODER` bits 11:10 = `10`.
- ☐ B1 steps LD2 through the level table, one level per press.
- ☐ No `HAL_Delay()` anywhere.
- ☐ Live Watch shows `period_us` = 1000 and `high_us` = 120 at level 120.
- ☐ Capture interrupt on channel 1 only (`HAL_TIM_IC_Start(..., TIM_CHANNEL_2)`); both subtractions cast to `uint16_t`.
- ☐ Prediction table filled before measuring; measured values match.

Individual oral check

- Why does `CCR1` = 999 give 99.9 % and not 100 %?
- TIM2 makes a 1 kHz PWM and a 1 ms tick at once. Why does that work?
- Why does the level table not use equal steps such as 0, 250, 500, 750, 1000?
- Who writes `TIM4->CCR2`, and at exactly which moment?
- Why is channel 2 *indirect*? What would a direct channel 2 listen to?
- Captures `64000`, then `1500`: what is `period_us`, and why is the cast needed?

Submit

Submit `main.c` as it is at the end of the session (with the prediction table in `USER CODE BEGIN Header`), `B_levels.png` and `D_measure.png`.

Marks (10): pre-lab quiz on CATS 1 · A dimmed LD2 with SFR evidence 2 · B brightness levels with B1 2 · C captures arrive 1 · D measurement matches the prediction 2 · oral check (one question) 2. The extension is optional and not graded.

If you are stuck

Walk the signal from left to right and fix the **first** silent stage: `TIM2->CCR1` value → `TIM2->CNT` moving → PA5 in alternate function (`MODER` bits 11:10 = `10`) → jumper on D13 and D10 → `TIM4->CNT` moving → `TIM4->CCR1` changing → `CC1IF` in `TIM4->SR` → NVIC enabled → breakpoint in the capture callback → `period_us`. **LD2 stays dark?** PA5 is still `GPIO_Output` in the `.ioc` (step 3), or `HAL_TIM_PWM_Start()` is missing. **LD2 full on all the time?** `CCR1` 1000, or the `TIM2->CCR1 = 120;` line is missing. **`g_ms` grows about 5 times faster than real time?** The Clock Configuration tab is still on the PLL at 84 MHz (step 2). **B1 does nothing?** EXTI line[15:10] is not enabled in NVIC (step 6). **One press skips two levels?** The 50 ms `last_press_ms` guard is missing. **`TIM4->CCR1` changes, but the callback never runs?** TIM4 global interrupt is not enabled in NVIC. **`period_us` stays 0?** The jumper is loose, or `HAL_TIM_IC_Start_IT()` is missing. **`high_us` is 0 or larger than `period_us`?** Channel 2 is direct instead of indirect, or its polarity is not falling, or `HAL_TIM_IC_Start(&htim4, TIM_CHANNEL_2)` is missing. **`period_us` is a different number at every breakpoint stop?** The timers keep running while the CPU is stopped. Read them in Live Watch. **Live Watch empty?** Add the `liveWatch` line, then restart the debugger. **LD2 dark and `g_ms` frozen after you wrote 999 into `ARR`?** `CNT` was above 999 and is counting on to the end of its 32-bit range, about 71 minutes. Pause and write 0 into `TIM2->CNT`.